

Informe de Mantenimiento - EPN Event Manager

INTEGRANTES:

- Maldonado Shirley
- Jarrin Scarleth
- Guamán Jonathan
- Sigcha Christian

1. Alcance del informe

Este informe documenta únicamente los mantenimientos aplicados al Hub central epn-event-manager. Los CRUDs externos se consideran clientes de prueba para verificar que el Hub recibe, valida, clasifica y persiste eventos correctamente.

2. Resumen de mantenimientos implementados

Tipo	Diagnóstico	Intervención	Justificación
Correctivo	Los eventos DELETE no se persistían correctamente y algunos registros llegaban incompletos.	Se corrigió la entidad/flujo de delete_events y se aseguró que los eventos DELETE tengan campos obligatorios como source, entity, action, title, description, payload y fecha.	Corrige un fallo real de funcionamiento: el sistema perdía eventos y la trazabilidad del CRUD quedaba incompleta.
Adaptativo	El Hub debía recibir eventos desde varios clientes y usar formatos compatibles con integraciones actuales.	Se habilitó CORS para los puertos de los clientes, se normalizaron fechas ISO 8601 y se activó el ValidationPipe global.	Adapta el sistema al nuevo entorno: varios CRUDs y frontends consumiendo el mismo Hub.
Perfectivo	El sistema funcionaba, pero era poco útil para análisis porque faltaban métricas y ordenamiento claro.	Se agregaron estadísticas, consulta consolidada y ordenamiento por fecha real de	Mejora la experiencia y el valor del sistema sin corregir necesariamente un error crítico.

		eventos.	
Preventivo	Las validaciones estaban mezcladas o podían llegar tarde en el flujo.	Se movieron reglas de entrada al DTO y se dejó al service concentrado en lógica de negocio.	Previene fallos futuros, separa responsabilidades y evita que datos inválidos lleguen al servicio.

3. Diagnóstico, intervención y justificación por mantenimiento

Correctivo

- Diagnóstico: El diseño separaba eventos en tablas create, update, delete y query. El problema más grave era que DELETE no quedaba registrado de forma confiable, por lo que al eliminar un elemento desde un cliente se perdía la evidencia del evento.
- Antes: El flujo no tenía una persistencia completa para DELETE o no respetaba la estructura que sí tenían CREATE/UPDATE/QUERY.
- Después: Se corrigió la persistencia de DELETE y se garantizó que el evento eliminado quede en delete_events con payload y fecha.
- Justificación: Es correctivo porque soluciona un fallo real que afectaba la estabilidad y la trazabilidad del sistema.

Adaptativo

- Diagnóstico: El Hub debía trabajar con varios clientes locales en distintos puertos, por ejemplo 4000, 4001, 4002 y 4003. Además, las fechas debían ser entendibles e integrables.
- Antes: CORS permitía pocos orígenes y algunos clientes eran bloqueados por el navegador.
- Después: Se amplió CORS en main.ts y se estandarizó el uso de fechas tipo ISO 8601.
- Justificación: Es adaptativo porque ajusta el sistema al nuevo entorno de integración del taller.

Perfectivo

- Diagnóstico: El Hub guardaba eventos, pero faltaban reportes o métricas para interpretar la información.
- Antes: La consulta de eventos no ofrecía suficiente valor para análisis.
- Después: Se incorporaron métricas/estadísticas y ordenamiento por fecha de eventos, permitiendo ver el comportamiento de los CRUDs.
- Justificación: Es perfectivo porque mejora una funcionalidad existente y aporta valor agregado.

Preventivo

- Diagnóstico: El sistema podía recibir datos inválidos o campos de más, y si las validaciones estaban en el service se mezclaban responsabilidades.

- Antes: Validaciones incompletas o ubicadas en la lógica del servicio.
- Después: Se usan DTOs con validaciones y ValidationPipe con whitelist, forbidNonWhitelisted y transform.
- Justificación: Es preventivo porque reduce riesgos futuros y mantiene una arquitectura más limpia.

4. Prueba de vida

- Levantar epn-event-manager en <http://localhost:3000>.
- Verificar /health para confirmar conexión del Hub y SQLite.
- Levantar los CRUDs/clientes y crear, consultar, actualizar o eliminar registros.
- Abrir /events y comprobar que se registran eventos por source, entity, action, payload y fecha.

5. Mejoras futuras sugeridas

Mejora futura	Tipo asociado	Por qué sería útil
Leer CORS desde variables de entorno (.env).	Adaptativo / Preventivo	Evita modificar código cada vez que cambie un puerto o cliente.
Migrar de cuatro tablas separadas a una tabla única events con campo action.	Perfectivo / Preventivo	Simplifica consultas, reduce duplicación y mejora mantenibilidad.
Agregar paginación a GET /events.	Perfectivo / Preventivo	Evita sobrecarga cuando existan muchos eventos.
Agregar filtros por source, entity, action y fecha.	Perfectivo	Facilita análisis y demostraciones por CRUD.
Crear pruebas automáticas para CREATE, UPDATE, DELETE, QUERY, health y stats.	Preventivo	Evita que futuros cambios rompan la persistencia o validación.
Mejorar logs y manejo de errores.	Preventivo	Permite diagnosticar fallos de SQLite, DTOs inválidos o eventos rechazados.
Normalizar completamente los campos de fecha entre tablas.	Adaptativo / Preventivo	Evita inconsistencias entre recorded_at, event_date, _eventDate u otros nombres.

6. Conclusión

Con los cambios realizados, epn-event-manager deja de ser solo un receptor básico y pasa a ser un Hub más estable, integrable, verificable y mantenible. Los CRUDs no son el foco del mantenimiento, pero sirven para demostrar que el Hub corregido recibe y persiste eventos correctamente.